copilot-instructions.md 配置指南

> 日期：2026-04-03
>
> 适用：GitHub Copilot（VS Code）

一、文件概述

`copilot-instructions.md` 是 GitHub Copilot 的工作区级系统提示，相当于给 Copilot 的"行为守则"。Copilot 在每次会话中都会自动加载这个文件的内容，并将其作为上下文前缀注入模型。

文件位置

```
<workspace-root>/
└── .github/
    └── copilot-instructions.md    ← 唯一有效路径
```

> 注意：必须是 `.github/copilot-instructions.md`，其他位置（如根目录）不会生效。

二、生效范围与加载规则

条件	说明
适用模式	Ask / Edit / Agent 三种模式均生效
适用模型	所有 Copilot 支持的模型（Claude、GPT-4o、Gemini 等）
加载时机	每次对话开始时自动注入
多 workspace	每个 workspace 独立加载各自的文件
子目录	不支持按子目录分割（整个 workspace 共用一份）
文件大小限制	建议 8KB 以内，过长会被截断

三、文件结构最佳实践

3.1 推荐结构

```
# Workspace Instructions
```

```
## 项目背景
（一句话说明项目是什么）

## 技术栈
（列出主要语言、框架、版本）

## 编码规范
（命名约定、格式要求等）

## 工作流约定
（固定的操作流程，如文档生成、测试流程等）

## 禁止行为
（明确告知 Copilot 不应做什么）
```

3.2 写作原则

- **命令式语气**：用"必须"、"不得"、"始终"，而非"应该"、"建议"
- **具体优于抽象**：给出具体路径、命令、示例，而非模糊描述
- **短句优于长段**：模型对列表和表格的理解优于大段文字
- **聚焦约束**：只写 Copilot 需要遵守的规则，不写背景知识

四、配置项详解

4.1 项目背景

```
## 项目背景
这是一个多模块 monorepo 项目，包含：
- `backend/`：Dart 服务端（shelf 框架）
- `frontend/`：Flutter 移动客户端
- `ibookjoy.com/`：Next.js 书评网站
- `sangogame.com/`：Next.js 游戏网站
```

作用：帮助 Copilot 理解项目结构，避免跨模块混淆技术栈。

4.2 技术栈声明

```
## 技术栈

| 模块 | 语言 | 框架 | 版本 |
|------|------|------|------|
| backend | Dart | shelf | 3.x |
| frontend | Dart | Flutter | 3.x |
| ibookjoy.com | TypeScript | Next.js 15 | App Router |
```

作用：防止 Copilot 使用过时 API 或错误框架的写法。

4.3 编码规范

```
## 编码规范

### 通用
- 文件编码：UTF-8
- 缩进：2 空格（所有语言）
- 不得添加不必要的注释和文档字符串

### Dart
- 类名：PascalCase
- 变量/函数：camelCase
- 私有成员加 _ 前缀

### TypeScript
- 使用 type 而非 interface（除非需要 extends）
- 禁止使用 any，用 unknown 替代
- 组件文件名：PascalCase.tsx
```

4.4 工作流约定

工作流是 `copilot-instructions.md` 最有价值的配置项，用于固化团队约定的操作流程。

```
## 文档生成工作流

每当生成 Markdown 文档，必须按以下步骤执行：

1. 将 .md 文件写入 `docs/<module>/` 对应目录
2. 在 workspace 根目录执行：
   ```bash
   bash tools/md_to_pdf.sh docs/<module>/<name>.md
   ```
3. 确认 `pdfdocs/<module>/<name>.pdf` 已生成后再报告完成

模块映射：
- backend → docs/backend/
- frontend → docs/frontend/
- ibookjoy.com → docs/ibookjoy/
```

4.5 禁止行为

```
## 禁止行为
```

```
- 不得创建未被要求的文档或总结文件
- 不得在未读取文件的情况下修改文件内容
- 不得使用 rm -rf 等破坏性命令，须询问用户确认
- 不得 git push --force，须询问用户确认
- 不得猜测 API URL，只使用代码中已有的地址
- 修改共享配置文件前必须询问用户
```

4.6 安全约束

```
## 安全要求

- 不得将密钥、Token、密码硬编码在代码中
- 敏感配置必须通过环境变量读取
- SQL 查询必须使用参数化查询，禁止字符串拼接
- 用户输入必须在系统边界处进行验证
```

五、进阶：`.instructions.md` 细粒度配置

除了全局的 `copilot-instructions.md` ，VS Code 还支持更细粒度的指令文件：

文件格式

```
---
applyTo: "**/*.dart"
---
# Dart 专属规则

- 优先使用 async/await 而非 .then()
- StreamController 必须在 dispose 中关闭
```

applyTo 模式示例

模式	说明
`**/*.dart`	所有 Dart 文件
`frontend/**`	frontend 目录下所有文件
`**/*.{ts,tsx}`	所有 TypeScript 文件
`**/test/**`	所有测试文件
`**`	所有文件（等同于全局）

存放位置

```
.github/
└── instructions/
    ├── dart.instructions.md        # applyTo: **/*.dart
    ├── typescript.instructions.md  # applyTo: **/*.{ts,tsx}
    └── tests.instructions.md       # applyTo: **/test/**
```

六、与其他配置文件的关系

文件	工具	作用域
`.github/copilot-instructions.md`	Copilot	整个 workspace
`.github/instructions/*.instructions.md`	Copilot	按文件类型/目录
`CLAUDE.md`	Claude Code CLI	整个项目
`AGENTS.md`	OpenAI Codex CLI	整个项目

这三类文件互不干扰，可以同时维护以支持不同工具。

七、本项目当前配置（iwithyou）

当前 `.github/copilot-instructions.md` 已配置：

- **Markdown 文档生成工作流**：自动 md → pdf 转换流水线
- **模块目录映射**：6 个模块各自的 docs 和 pdfdocs 路径
- **操作约束**：禁止手动调用转换脚本、自动创建 pdfdocs 目录

建议补充：

- [ ] 各模块技术栈声明（防止 Copilot 混淆 Dart/TS 规范）
- [ ] 编码规范（命名约定、禁用 any 等）
- [ ] 安全约束（禁止硬编码密钥）
- [ ] 破坏性操作确认要求（git push --force、rm -rf）

八、配置验证

修改 `copilot-instructions.md` 后，可以用以下方式验证是否生效：

1. 在 Copilot Chat 中询问："你有哪些工作约定？"
2. Copilot 应能复述文件中的关键规则

3. 如果没有响应，检查文件路径是否正确（必须是 `.github/copilot-instructions.md` ）
4. 重新打开 VS Code 窗口以确保重新加载

本指南基于 *GitHub Copilot 2025-2026* 版本行为，功能迭代较快，建议定期核实官方文档。